MODULE

# 22

# FastAPI in depth

Design production APIs: validation, dependency injection, errors, middleware, settings and async.

YOU WILL LEARN

| | |
|---|---|
| Pydantic models | `01_pydantic_models.py` |
| Routers & dependencies | `02_routers_and_dependencies.py` |
| Errors, middleware & CORS | `03_errors_middleware_cors.py` |
| Settings, lifespan & async | `04_settings_lifespan_async.py` |
| Interview Q&A | `INTERVIEW.md` |

WHY VALIDATION

# Never trust the client

THE PROBLEM

A mobile app sends **{"qty": "-5", "email": "lol"}**. Without validation you store garbage, or a negative order refunds money.

THE IDEA

**Describe the data once with Pydantic. FastAPI validates every request, converts types, and answers 422 with clear errors.**

TRY IT PYDANTIC

# Models that validate and convert

```python
from pydantic import BaseModel, Field, ValidationError, field_validator

class OrderIn(BaseModel):
    email: str = Field(pattern=r".+@.+\..+")
    item: str = Field(min_length=1, max_length=50)
    qty: int = Field(gt=0, le=100)

    @field_validator("item")
    @classmethod
    def clean(cls, v): return v.strip().lower()

print(OrderIn(email="v@ex.com", item="  Lamp ", qty="3"))   # "3" -> 3

try:
    OrderIn(email="lol", item="", qty=-5)
except ValidationError as e:
    for err in e.errors():
        print(err["loc"][0], "->", err["msg"])
```

OUTPUT

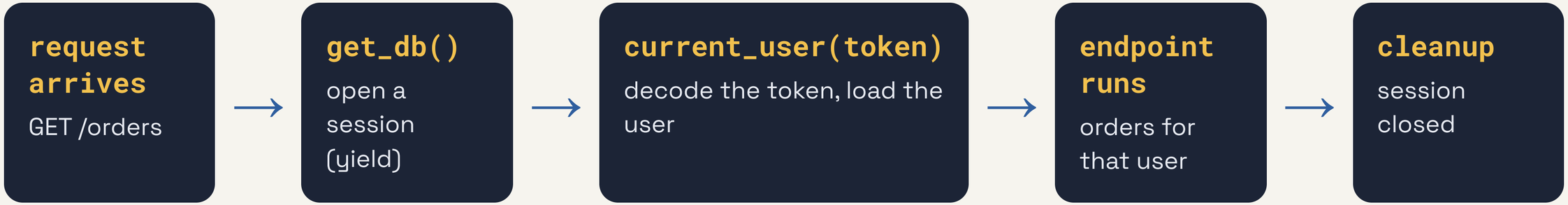
```
email='v@ex.com' item='lamp' qty=3
email -> String should match pattern
'.+@.+\..+'
item -> String should have at least 1
character
qty -> Input should be greater than 0
```

1. Types are **converted** (”3” → 3) and constraints checked.
2. All errors at once, with the field name.
3. Separate models for **input** and **output** (never return password hashes).

PICTURE IT DEPENDENCY INJECTION

# Depends(): FastAPI hands you what you need

🔌 Instead of each endpoint creating its own DB session or checking the token, it **declares** what it needs. FastAPI builds it, passes it in, and cleans up afterwards. Tests can **swap** any dependency.



TRY IT DEPENDENCY INJECTION

# Chained dependencies + override in tests

```python
from typing import Annotated
from fastapi import Depends, FastAPI, Header, HTTPException
from fastapi.testclient import TestClient

app = FastAPI()
USERS = {"t-vishal": {"name": "vishal", "role": "admin"}}

def current_user(authorization: Annotated[str | None, Header()] = None):
    user = USERS.get((authorization or "").removeprefix("Bearer "))
    if not user:
        raise HTTPException(401, "not logged in")
    return user

def admin(user: Annotated[dict, Depends(current_user)]):
    if user["role"] != "admin":
        raise HTTPException(403, "admins only")
    return user

@app.delete("/books/{book_id}", status_code=204)
def delete_book(book_id: int, user: Annotated[dict, Depends(admin)]): ...

c = TestClient(app)
print(c.delete("/books/1").status_code)
print(c.delete("/books/1", headers={"Authorization": "Bearer t-vishal"}).status_code)
```

OUTPUT

```
401
204
```

1. **admin** depends on **current_user**: a chain.
2. **401** = who are you? **403** = I know you, but no.
3. Tests swap any of them with **dependency_overrides**.

STEP BY STEP PROJECT STRUCTURE

# A FastAPI project that scales

| FILE | JOB |
| --- | --- |
| `main.py` | create the app, include routers, middleware, lifespan |
| `routers/orders.py` | `APIRouter(prefix="/orders", tags=["orders"])` : thin endpoints |
| `schemas.py` | Pydantic in/out models |
| `services/` | business rules, no HTTP code |
| `deps.py` | get_db, current_user, pagination params |
| `config.py` | `Settings(BaseSettings)` from environment variables |

> Endpoints stay thin: validate → call a service → return a schema.

TRY IT ERRORS AND MIDDLEWARE

# One place for errors, one for every request

```python
import uuid
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.testclient import TestClient

app = FastAPI()

class OutOfStock(Exception):
    def __init__(self, item): self.item = item

@app.exception_handler(OutOfStock)
def out_of_stock(request: Request, exc: OutOfStock):
    return JSONResponse({"detail": f"{exc.item} is sold out"}, status_code=409)

@app.middleware("http")
async def request_id(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Request-ID"] = str(uuid.uuid4())[:8]
    return response

@app.post("/orders")
def order(): raise OutOfStock("lamp")

r = TestClient(app).post("/orders")
print(r.status_code, r.json(), "has id:", "x-request-id" in r.headers)
```

OUTPUT

```
409 {'detail': 'lamp is sold out'}
has id: True
```

1. Domain errors → HTTP codes in **one** place.
2. Middleware wraps **every** request: IDs, timing, logging.
3. CORS is middleware too: **CORSMiddleware(allow_origins=[...])**.

COMMON MISTAKE CORS

# ”Fixing” CORS with a wildcard

✗ WRONG

```
app.add_middleware(CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,     # with cookies!
    allow_methods=["*"])
```

Any website could call your API from a victim’s browser. (Browsers reject * with credentials anyway, so it also doesn’t work.)

✓ RIGHT

```
app.add_middleware(CORSMiddleware,
    allow_origins=["https://app.shop.de"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PATCH"],
    allow_headers=["Authorization", "Content-Type"])
```

CORS is a **browser** rule. List the exact frontend origins. It doesn’t protect against curl or other servers.

TRY IT SETTINGS AND LIFESPAN

# Config from the environment, startup and shutdown

```python
import os
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str = "sqlite:///dev.db"
    debug: bool = False

os.environ["DEBUG"] = "true"           # e.g. set by Docker / Azure
settings = Settings()
print(settings)

@asynccontextmanager
async def lifespan(app):
    print("startup: open DB pool, HTTP client")
    yield
    print("shutdown: close them")

app = FastAPI(lifespan=lifespan)
with TestClient(app):
    print("serving requests...")
```

OUTPUT

```
database_url='sqlite:///dev.db'
debug=True
startup: open DB pool, HTTP client
serving requests...
shutdown: close them
```

1. **12-factor**: config in env vars, never in code.
2. **lifespan**: create shared clients once, close them cleanly.
3. Secrets come from the environment or a vault.

IN REAL JOBS FASTAPI

# FastAPI patterns you'll see at work

## Pagination dependency

```
def page(limit: int = Query(20, le=100), offset: int = 0):
    return {"limit": limit, "offset": offset}
```

## Background work

```
@app.post("/signup")
def signup(bg: BackgroundTasks): bg.add_task(send_mail)
```

## Response models hide fields

```
@app.get("/me", response_model=UserOut)
```

## Run in production

```
uvicorn app.main:app --workers 4
# or gunicorn -k uvicorn.workers.UvicornWorker
```

INTERVIEW SAY THIS IN THE INTERVIEW

# “What is dependency injection in FastAPI? Why is it useful?”

- Endpoints **declare** what they need with **Depends()**; FastAPI creates it per request and passes it in.
- Dependencies can depend on others (token → user → admin), and **yield** dependencies clean up afterwards.
- It removes duplication (auth, DB sessions, pagination) and keeps endpoints thin.
- In tests, **app.dependency_overrides** swaps the real DB or external clients for fakes.

> **Follow-up they ask next:** ”401 vs 403?” (401: not authenticated, send credentials; 403: authenticated but not allowed).

INTERVIEW SAY THIS IN THE INTERVIEW

# “How does FastAPI validate requests? What is 422?”

- Parameters and bodies are declared with **type hints** and **Pydantic** models.
- FastAPI parses path, query, headers and JSON, **converts types** and checks constraints.
- Invalid input gets a **422 Unprocessable Entity** with a list of field errors, before your code runs.
- The same models generate the **OpenAPI** docs at /docs.

> **Follow-up they ask next:** ”What is CORS and when do you need it?” (a browser rule: a page from origin A may call API origin B only if B allows A via CORS headers).

RECAP REMEMBER THESE

# Module 22 in 6 lines

1. **Pydantic** validates and converts; separate In/Out models.
2. **Depends()** for DB sessions, auth, pagination; chain them.
3. 401 = who are you, 403 = not allowed, 422 = bad input.
4. Exception handlers map domain errors; middleware wraps every request.
5. CORS: exact origins only.
6. **BaseSettings** for env config; **lifespan** for startup/shutdown.

PRACTICE NOW

**Extend 02_routers_and_dependencies.py: add PATCH /books/{id} for admins only, with a test.**